



Выпускная квалификационная работа

Разработка веб-приложения для организации личного файлового хранилища

LocalCloud — self-hosted облачное хранилище

Выполнил: **Магомедов Имран Борисович**, гр. ПИЖ-б-о-22-1
Направление: **09.03.04 Программная инженерия**
Руководитель: **к.т.н., доцент Самойлов Ф. В.**
Ставрополь, 2026

Актуальность



- **Рост личных данных:** фото/видео, документы, архивы — объём постоянно растёт.
- **Публичные облака** (Google Drive, Dropbox, Я.Диск) — подписка, рост платежей, риски: блокировки, смена тарифов, контроль над данными у провайдера; после 2022 г. — проблемы оплаты.
- **Self-hosted** (Nextcloud, ownCloud, Seafile) решает приватность, но рассчитан на мощные серверы: файловый трафик идёт через приложение → высокая нагрузка CPU/RAM.
- **Типовой доступный сервер** частного пользователя — бюджетный VPS: 1 CPU / 1 ГБ ОЗУ.
 - Инженерная задача: передавать и обрабатывать данные, многократно превышающие память сервера.

Цель и задачи



Цель

Разработать веб-приложение LocalCloud: личное файловое хранилище с загрузкой/скачиванием больших файлов, иерархией, разграничением доступа и публикацией по ссылкам, сохраняющее работоспособность на сервере 1 CPU / 1 ГБ ОЗУ.

- Проанализировать предметную область и аналоги, сформировать требования.
- Спроектировать архитектуру по С4, БД, REST API, подсистемы хранения, задач и безопасности.
- Реализовать серверную и клиентскую части и средства развёртывания.
- Провести модульное, интеграционное, нагрузочное тестирование и тестирование безопасности.
- Выполнить технико-экономическое обоснование и рассмотреть охрану труда.

Заинтересованные лица (акторы)

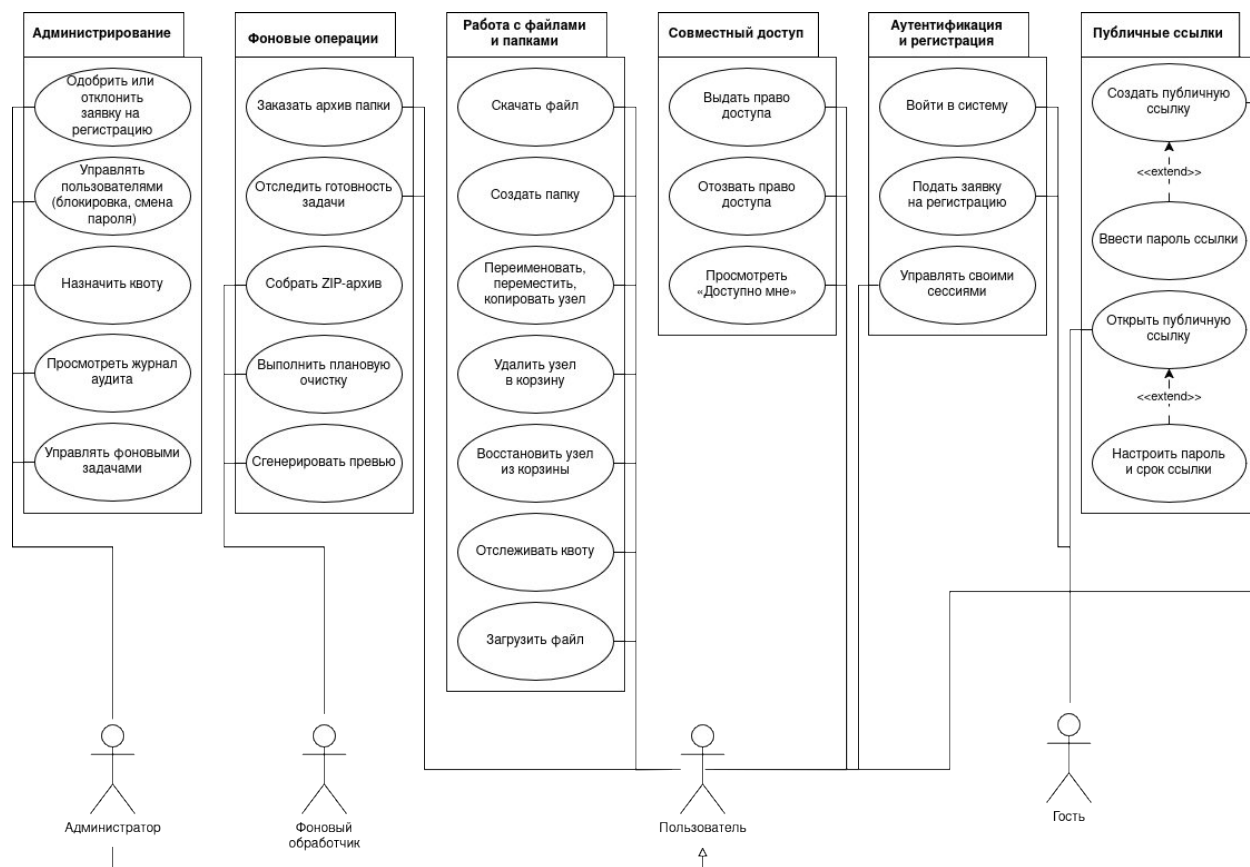


■ **Гость:** вход, заявка на регистрацию, доступ по публичной ссылке (просмотр / скачивание / загрузка, возможно по паролю).

■ **Пользователь:** свои файлы, папки и корзина; выдача доступа другим; публичные ссылки; квота и сессии.

■ **Администратор:** учётные записи и заявки, блокировки, квоты, журнал аудита, фоновые задачи.

■ **Фоновый обработчик** (системный актор): архивы, превью, плановая очистка из очереди задач.



Основной функционал



- **Файлы и папки:** дерево произвольной вложенности, перемещение, копирование, переименование, поиск.
- **Многочастичная загрузка** напрямую в объектное хранилище по presigned-URL (мимо backend).
- **Скачивание архивами:** упаковка папок/наборов в ZIP фоновым процессом.
- **Корзина:** мягкое удаление, восстановление, автоочистка.
- **Общий доступ:** права между пользователями (5 уровней, наследование) и публичные ссылки (срок, пароль).
- **Квоты, превью** (изображения, PDF, видео), **аутентификация** (JWT, регистрация по заявке), **админ-панель**.

Нефункциональные требования



- **Производительность:** работа на 1 CPU / 1 ГБ ОЗУ при 3–5 пользователях; размер файла не ограничен памятью сервера.
- **Безопасность:** JWT в httpOnly-cookie с ротацией, пароли только хэшами, проверка прав на каждую операцию, защита от перебора.
- **Надёжность:** перезапуск без потери данных, повтор фоновых задач с нарастающей задержкой, миграции схемы.
- **Интерфейс:** адаптивный SPA, тёмная/светлая темы, отзывчивость.
- **Развёртывание и сопровождение:** запуск одной командой Docker Compose, конфигурация под ресурсы хоста.

Архитектура системы (C4: контейнеры)



- Многоуровневая клиент-серверная архитектура; модель **C4**.
- **SPA** (React) → только через шлюз.
- **nginx-шлюз** — единая точка входа: /, /api, бакеты.
- **API** (FastAPI) — метаданные, права, координация загрузок.
- **Worker** — ресурсоёмкие и плановые операции.
- **PostgreSQL 16** — метаданные и очередь задач.
- **MinIO** — S3-хранилище содержимого.
- Ключ: файловый трафик идёт SPA → шлюз → MinIO в обход API.

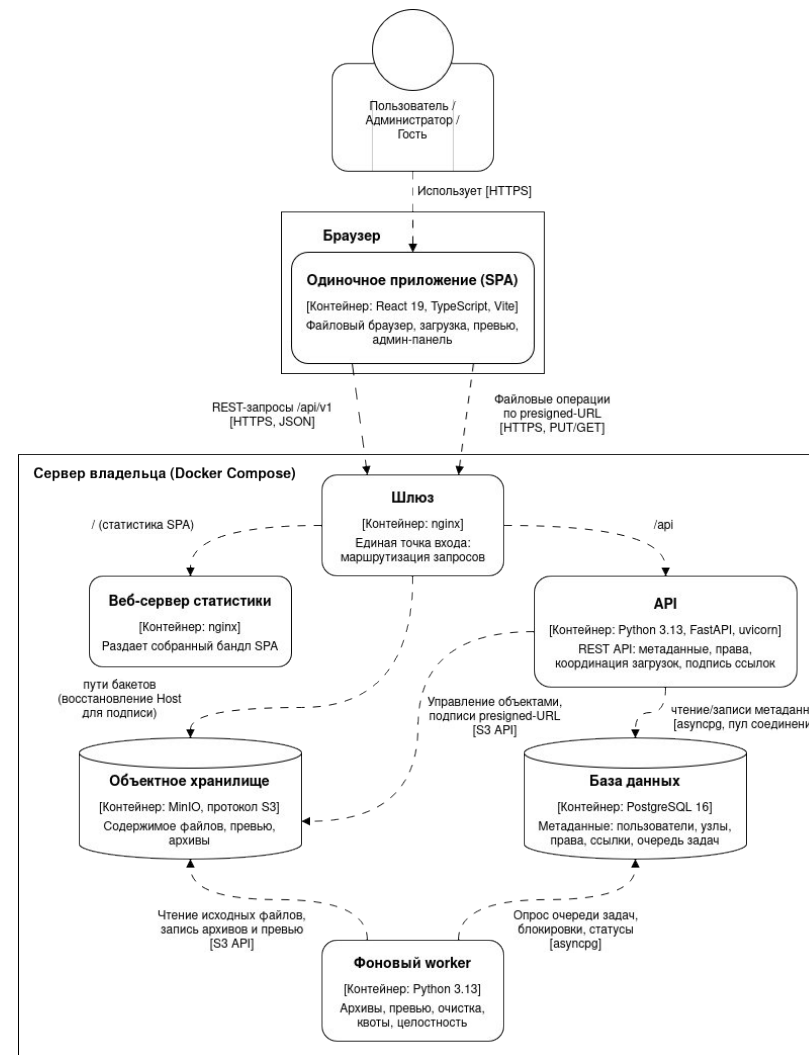


Рисунок 2.2 — Диаграмма контейнеров (C4, C2)

Слои серверного приложения (C4: компоненты)



- **Маршрутизаторы (api):** валидация схем, права, формирование ответа; знают только о сервисах.
- **Доменные сервисы (services):** бизнес-логика прецедентов; оркеструют репозитории и хранилище.
- **Репозитории (repositories):** инкапсулируют SQL; рекурсивные CTE для деревьев и прав.
- **Unit of Work:** владеет сессией БД, явная фиксация, авто-откат при выходе без commit.
- **Модели (models):** декларативные классы SQLAlchemy 2 (Mapped).
- **Storage-фасад:** единый интерфейс к MinIO (бакеты, объекты, multipart, presigned).
- Зависимости строго к нижним слоям — циклов нет.

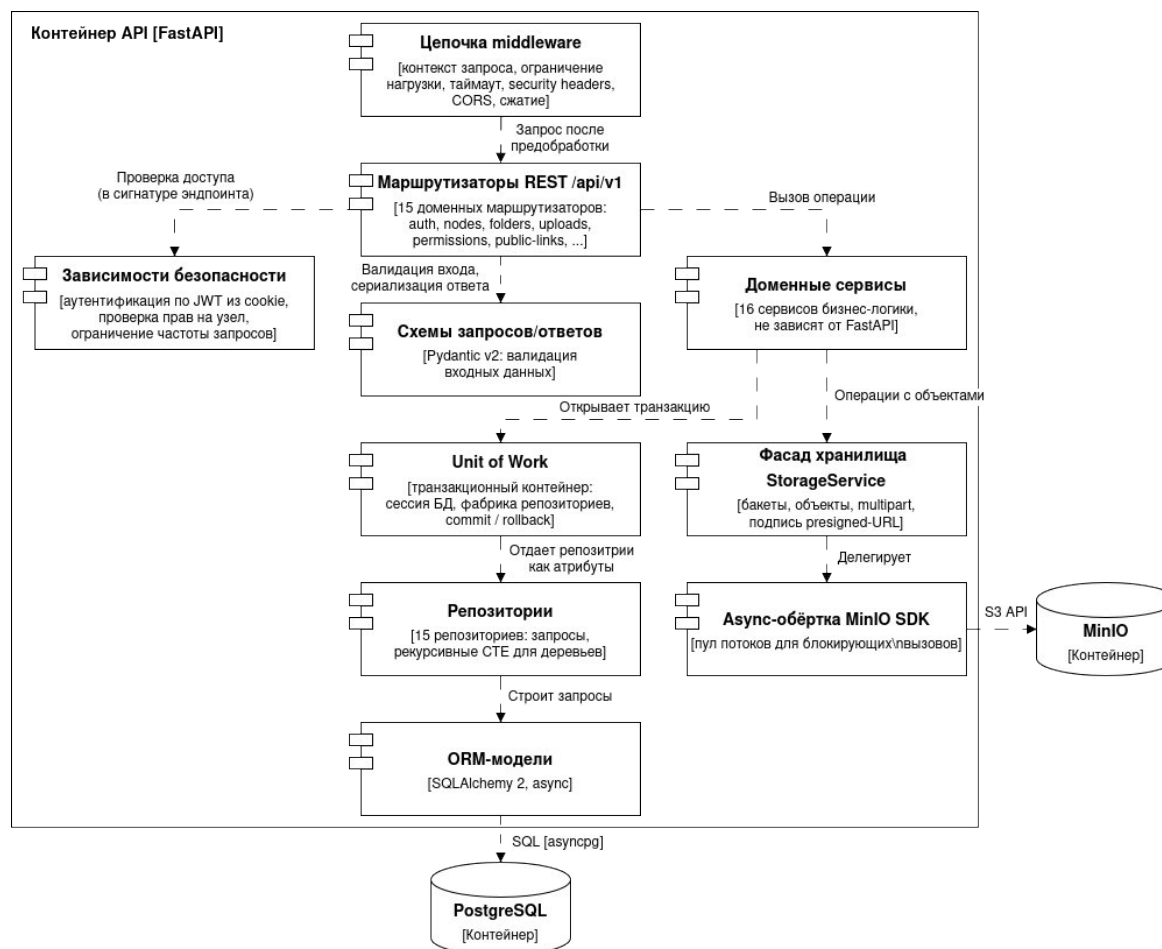


Рисунок 2.3 — Диаграмма компонентов (C4, C3)

Паттерны проектирования



- **Repository** — изоляция бизнес-логики от SQL/ORM.
- **Unit of Work** — транзакционный контейнер с явной фиксацией.
- **Facade** — StorageService как единый интерфейс к хранилищу.
- **Adapter** — асинхронная обёртка блокирующего MinIO SDK поверх пула потоков.
- **Dependency Injection** — зависимости через механизм FastAPI (развязка слоёв, подмена в тестах).
- **Command + Registry** — обработчики фоновых задач в реестре по типу задачи.
- **Proxy** — ленивый прокси настроек (чтение конфигурации при первом обращении).

Проектирование БД · инфологическая модель



- **14 таблиц**, нормализация до 3НФ, суррогатные ключи UUID.
- Три контура: **файловая иерархия** (узлы/файлы/папки), **доступ** (права/ссылки/сессии), **служебный** (загрузки/задачи/квоты/аудит).
- Узел file_system_nodes + спутники files / folders («один-к-одному») — общие операции единообразны.
- Метаданные (PostgreSQL) и содержимое (MinIO по ключу объекта) разделены уже на концептуальном уровне.

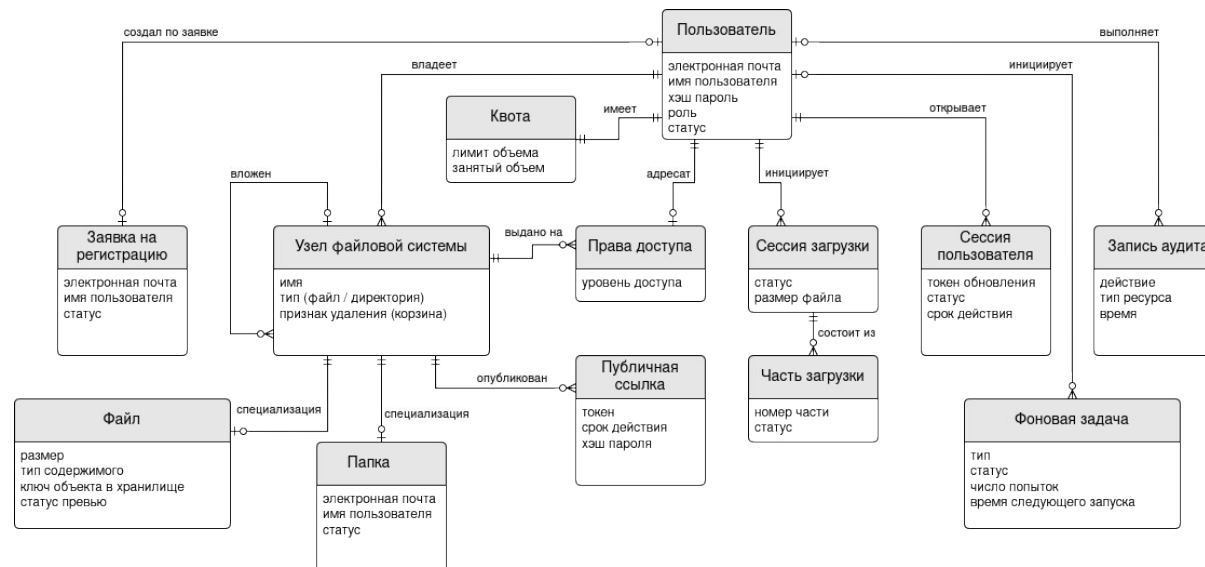


Рисунок 1.2 — Концептуальная модель данных (ER)



Backend

- Python 3.13, FastAPI, uvicorn
- SQLAlchemy 2 (async) + asyncpg
- Alembic, Pydantic v2
- python-jose (JWT), passlib (bcrypt/argon2)
- MinIO SDK, Pillow, PyMuPDF

Frontend

- React 19, TypeScript 6
- Vite 8, React Router 7
- TanStack Query / Virtual
- Tailwind CSS 4, Radix UI / shadcn
- Axios, lucide-react, Sonner

Хранилище / Инфра

- PostgreSQL 16 (метаданные)
- MinIO (S3-совместимое)
- Docker Compose (6 сервисов)
- nginx (шлюз + SPA)
- uv, pytest, Vitest, ESLint, ruff

Структура проекта (монорепозиторий)



```

LocalCloud/
├── backend/
│   ├── api/
│   ├── services/
│   ├── database/
│   ├── storage/
│   ├── workers/
│   └── tests/
├── frontend/
│   ├── src/
│   └── tests/
├── nginx/
├── scripts/
└── docker-compose.yml
    
```

FastAPI API + фоновый worker
 роутеры /api/v1 (15 групп)
 доменные сервисы (16)
 модели, репозитории (14), UoW, миграции
 фасад MinIO (presigned, multipart)
 диспетчер, реестр, планировщик
 unit + integration (pytest)
 SPA на React + Vite
 api, components, hooks, pages
 Vitest + Testing Library
 шлюз / reverse-proxy
 генератор .env под ресурсы хоста
 весь стек (6 сервисов)

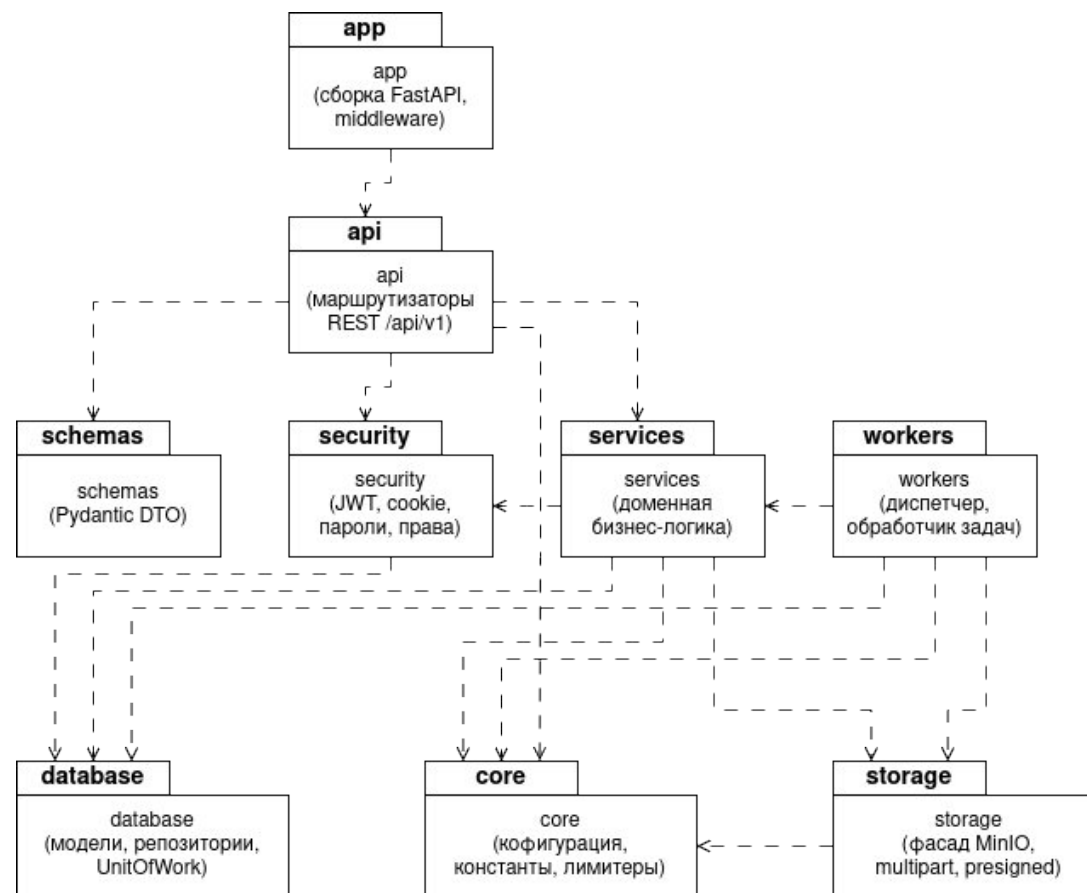


Рисунок 3.3 — Диаграмма пакетов

Прошедшие проектные этапы и процесс



- Жизненный цикл — 5 этапов: анализ и проектирование, кодирование, тестирование, документация, развёртывание.
- **Git Flow:** main / develop, ветки feature/* · release/* · hotfix/*; свыше 80 коммитов.
- **Conventional Commits** + **SemVer** (теги v0.1.0-beta.1, v0.1.0, v0.1.1); CHANGELOG.
- Оценка трудоёмкости методом **PERT**; план — диаграмма Ганта.

LocalCloud — диаграмма Ганта разработки

Self-hosted облачное хранилище · февраль – июнь 2026

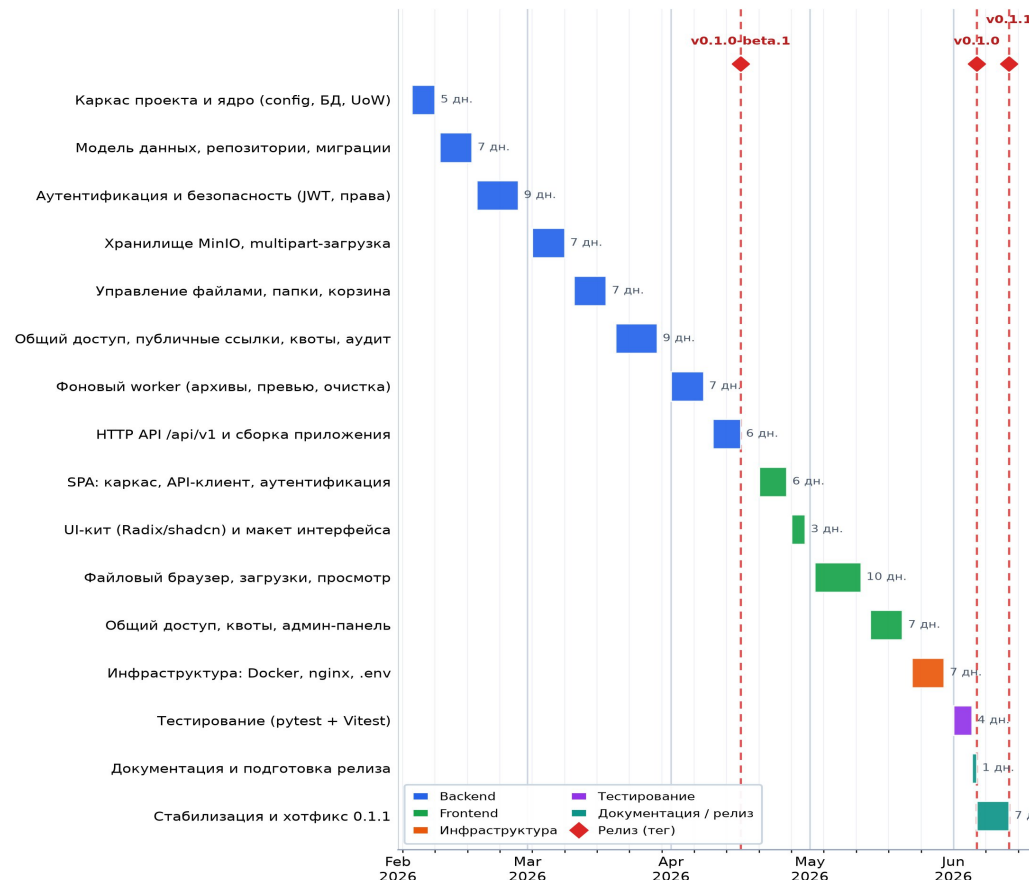


Рисунок 5.1 — Календарный план (Ганта)

Реализация · ключевые моменты



- **Загрузка мимо backend:** клиент пишет части напрямую в MinIO по presigned-URL (часть 8 МБ), сервер лишь координирует сессию и подписывает ссылки → память сервера не зависит от размера файла.
- **Наследование прав:** право на папку распространяется на поддерево; обход предков одним рекурсивным CTE вместо N+1 запросов.
- **Фоновый worker:** очередь задач в PostgreSQL, конкурентный захват через SELECT FOR UPDATE SKIP LOCKED, повторы с экспоненциальной задержкой, планировщик периодических задач.
- **Тихое продление сессии:** перехватчик axios обновляет токены при 401, остальные запросы ждут в очереди — без гонок.
- **Превью** изображений/PDF/видео и **упаковка ZIP** — потоково, без буферизации целиком в памяти.



Пример запроса · наследование прав (рекурсивный CTE)

SQL-запрос (PostgreSQL)

```
WITH RECURSIVE permission_ancestors AS (  
    SELECT id, parent_id FROM file_system_nodes  
    WHERE id = :node_id  
    UNION ALL  
    SELECT n.id, n.parent_id FROM file_system_nodes n  
    JOIN permission_ancestors a ON n.id = a.parent_id  
    WHERE n.is_deleted = false  
)  
SELECT * FROM node_permissions  
WHERE node_id IN (SELECT id FROM permission_ancestors)  
AND user_id = :user_id AND <активно>;
```

Код (SQLAlchemy 2, async)

```
ancestors = (  
    select(Node.id, Node.parent_id)  
    .where(Node.id == node_id)  
    .cte('permission_ancestors', recursive=True))  
parent = aliased(Node)  
ancestors = ancestors.union_all(  
    select(parent.id, parent.parent_id).where(  
        parent.id == ancestors.c.parent_id,  
        parent.is_deleted.is_(False)))  
stmt = select(NodePermission).where(  
    NodePermission.node_id.in_(select(ancestors.c.id)),  
    NodePermission.user_id == user_id, *active)
```

Эффективные права с учётом наследования — за один запрос (устранён антипаттерн N+1).



Пример запроса · захват задач очереди (SKIP LOCKED)

SQL-запрос (PostgreSQL)

```
SELECT * FROM background_tasks
WHERE status = 'PENDING'
  AND (scheduled_at IS NULL OR scheduled_at <= now())
  AND (locked_until IS NULL OR locked_until < now())
ORDER BY priority, scheduled_at, created_at
LIMIT :batch
FOR UPDATE SKIP LOCKED;
```

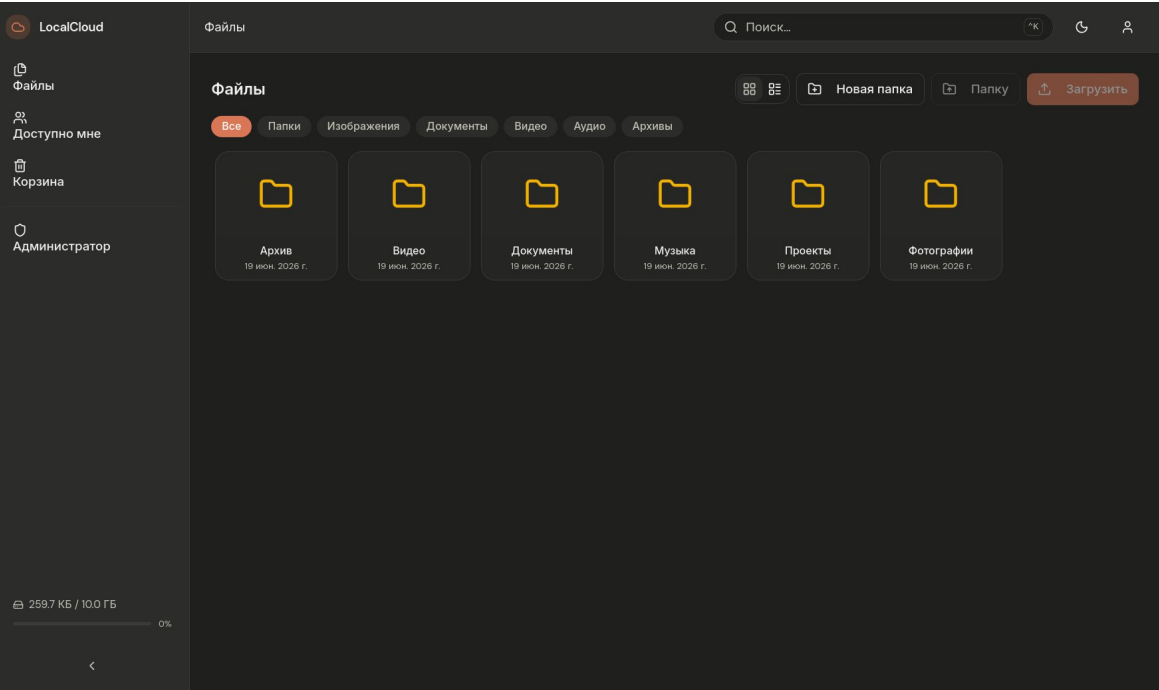
Код (SQLAlchemy 2, async)

```
stmt = (
    select(BackgroundTask)
    .where(*conditions)
    .order_by(
        priority_rank.asc(),
        BackgroundTask.scheduled_at.asc().nullsfirst(),
        BackgroundTask.created_at.asc())
    .limit(limit)
    .with_for_update(skip_locked=True))
tasks = await self.scalars_all(
    stmt, operation='lock_due_tasks')
```

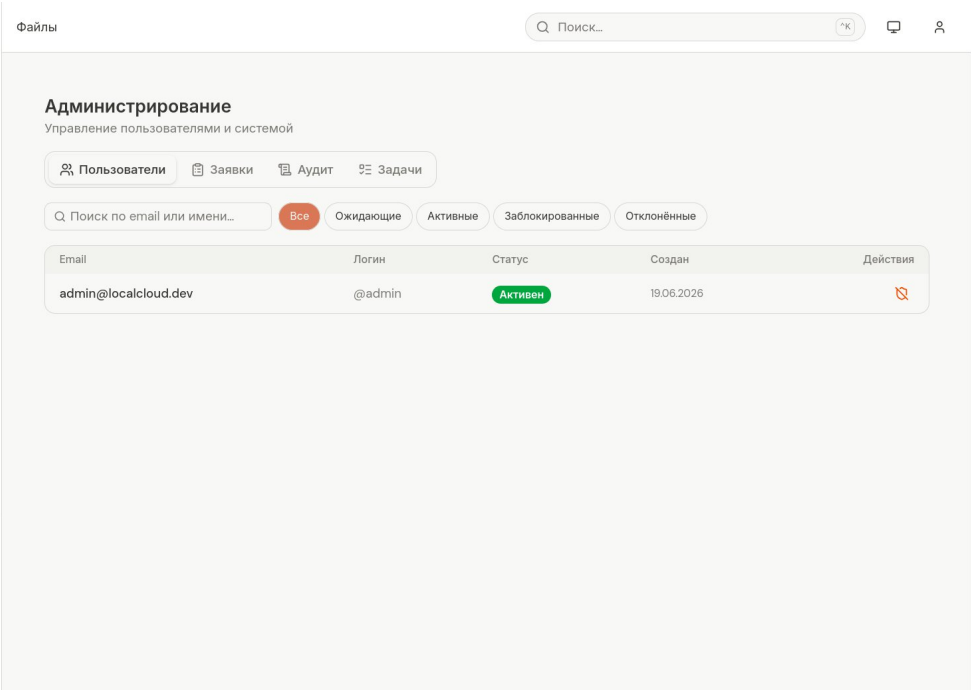
Несколько воркеров конкурентно берут задачи без распределённого координатора и брокера.

РАЗДЕЛ 3 · КЛИЕНТСКАЯ ЧАСТЬ

Пользовательский интерфейс



Файловый браузер



Админ-панель

Тестирование и покрытие



7679

автотестов — все
пройденны

6745

backend (6559 unit + 186
integ.)

934

frontend (Vitest)

98,6%

покрытие backend

- Три уровня: модульное, интеграционное (REST API через TestClient), системное; среда без внешних сервисов.
- Покрытие: backend **98,6 %** операторов, frontend **90,2 %** (пороги 85/80 % закреплены в конфигурации).
- Нагрузочное тестирование: p99 задержки ≤ 177 мс при требовании 2 с; управляемое поведение при перегрузке.
- Тестирование безопасности: JWT, права, защита от перебора, гигиена секретов.

Технико-экономическое обоснование



802 141 ₽

себестоимость
разработки

38 400 ₽

экономия владельца в
год

1,6 года

срок окупаемости

8,25

индекс доходности

- Трудоёмкость оценена методом **PERT** (5 этапов); объём кода $\approx$ 126 тыс. строк без тестов.
- Полная себестоимость — **802 141 ₽**, договорная цена с НДС — **1 203 211 ₽**.
- Продукт тиражируемый: чистый дисконтированный доход за 5 лет — **5,82 млн ₽** (консервативный прогноз).
- Качественные эффекты: приватность, контроль над данными, независимость от провайдера, вклад в open source.

Безопасность и охрана труда



- **Рабочее место** по ГОСТ 12.2.032-78: площадь $\geq 4,5 \text{ м}^2$, объём $\geq 15 \text{ м}^3$, высота стола 725 мм, регулируемое кресло.
- **Микроклимат** для категории работ Ia по СанПиН 1.2.3685-21 (температура, влажность, скорость воздуха).
- **Освещение:** совмещённое естественное и искусственное, нормированная освещённость рабочей зоны.
- **Электробезопасность:** помещение без повышенной опасности (ПУЭ).
- **Пожарная безопасность:** категория В4, класс пожара Е (СП 12.13130.2009).
- **Эргономика и режим труда:** регламентированные перерывы 10–15 мин при работе с ПК.

Заключение



- Все поставленные задачи решены; цель достигнута.
- Создан готовый к развёртыванию продукт: стек из 6 сервисов поднимается одной командой, конфигурация автоматически подбирается под ресурсы хоста.
- Ключевое решение — файловый трафик в обход backend — позволяет обслуживать файлы, размер которых многократно превышает ОЗУ сервера.
- Качество подтверждено: 7679 автотестов, покрытие 98,6 % / 90,2 %, нагрузочное и security-тестирование.
- Переиспользуемые решения: presigned multipart, очередь задач в СУБД, наследуемые права на дерево.



Спасибо за внимание!

Готов ответить на вопросы
github.com/magomedov-dev/LocalCloud